Object-Oriented Programming in C

Practical notes for a large C project: OOP concepts, module structure, headers, Makefiles, random-number generation and probability distributions.

1. OOP in C: the basic idea

C has no native classes, inheritance or virtual methods. However, its structs, functions, function pointers and separate compilation provide the primitives needed to build OOP-like designs.

OOP concept	Typical C equivalent
Class	struct + functions
Object	Instance of a struct
Constructor	init/create function
Method	Function receiving a pointer to the struct
Private fields	Opaque struct / implementation hidden in .c
Inheritance	Struct composition / embedding
Polymorphism	Function pointers / manual vtables
Interface	Header containing public declarations

2. Structs and methods

A C function can receive a pointer to the object. This pointer plays a role similar to C++'s implicit **this** pointer.

```
typedef struct {
    int age;
} Dog;

void Dog_bark(Dog *self)
{
    printf("Woof!\n");
}

int main(void)
{
    Dog dog = { .age = 5 };
    Dog_bark(&dog);
}
```

3. Constructors and destructors

```
void Dog_init(Dog *self, int age)
{
    self->age = age;
}

Dog dog;
Dog_init(&dog, 5);
```

For dynamically allocated objects, a create/destroy pair is a common convention.

```
Dog *dog = malloc(sizeof(*dog));
Dog_init(dog, 5);

/* ... */
```

```
free(dog);
```

4. Encapsulation with opaque structs

A header can expose only the existence of a type while keeping its fields private in the .c file. This is one of the most useful OOP-like techniques in C.

```
/* dog.h */
#pragma once

typedef struct Dog Dog;

Dog *Dog_create(int age);
void Dog_destroy(Dog *dog);
void Dog_bark(Dog *dog);
int  Dog_get_age(Dog *dog);
```

```
/* dog.c */
#include "dog.h"
#include <stdio.h>
#include <stdlib.h>

struct Dog {
    int age;
};

Dog *Dog_create(int age)
{
    Dog *dog = malloc(sizeof(*dog));
    dog->age = age;
    return dog;
}
```

5. Composition and inheritance-like designs

C has no built-in inheritance. Struct embedding/composition can provide an inheritance-like relationship.

```
typedef struct {
    int age;
} Animal;

typedef struct {
    Animal animal;
    const char *name;
} Dog;
```

The first member can sometimes be used for layout-compatible conversions, but this should be deliberate. In many designs, explicit composition is clearer than trying to imitate class inheritance.

6. Polymorphism with function pointers

```
typedef struct Animal Animal;

struct Animal {
    void (*speak)(Animal *self);
};

void dog_speak(Animal *self)
{
    printf("Woof!\n");
}

void cat_speak(Animal *self)
{
    printf("Meow!\n");
}
```

```
Animal dog = { .speak = dog_speak };
Animal cat = { .speak = cat_speak };

dog.speak(&dog);
cat.speak(&cat);
```

A struct containing function pointers is conceptually similar to a manual vtable. This is a common way to implement interfaces and dynamic dispatch in C.

7. Modules: .h and .c

For a large C project, the recommended model is generally one module consisting of a header (public interface) and a source file (implementation). Keep the pair together when they belong to the same module.

```
src/
■■■ agents/
■   ■■■ agents.c
■   ■■■ agents.h
■■■ engine/
■   ■■■ monte_carlo.c
■   ■■■ monte_carlo.h
■■■ random/
■   ■■■ rng.c
■   ■■■ rng.h
■   ■■■ distributions.c
■   ■■■ distributions.h
■■■ main.c
```

A top-level include/ directory is more useful when the project is a library and you need to distinguish public API headers from internal headers. For an application, keeping .h and .c together by module is often cleaner.

8. #pragma once

```c
/* header.h */
#pragma once

typedef struct {
    int x;
    int y;
} Point;
```

#pragma once belongs at the top of a header. It prevents the same header from being processed more than once in a compilation unit. It is not normally needed in .c files. Traditional include guards (#ifndef/#define/#endif) are the portable older alternative.

9. Repeated includes and executable overhead

Repeatedly including a guarded header does not create repeated copies of ordinary declarations in the executable. Function declarations in headers do not generate function bodies. Implementations normally live in .c files and are compiled/linked separately. Header processing can have compile-time cost, but #pragma once is handled efficiently by modern compilers.

A useful rule is that every header should be self-contained: if it needs a type or declaration from another header, include that header itself rather than relying on inclusion order elsewhere.

10. Standard math.h vs project math utilities

Avoid naming your own header **math.h**, because that collides conceptually (and potentially through include paths) with the standard C header. Prefer names such as **math_utils.h**, **numerical.h**, or a more specific module name.

```c
#include <math.h>          /* C standard math */
#include "math_utils.h"  /* project code */
```

11. What standard math.h provides

Typical functions include sqrt, pow, exp, log, log10, sin, cos, tan, asin, acos, atan, atan2, fabs, fmod, floor, ceil, round and trunc. For example, use exp(x) for e^x rather than pow(EULER, x).

```
#include <math.h>

double y1 = sqrt(x);
double y2 = exp(x);     /* e^x */
double y3 = log(x);     /* natural logarithm */
double y4 = atan2(y, x);
```

For GCC on Linux, mathematical functions commonly require linking with libm using **-lm**.

12. Euler's number and floating-point types

```
static const double EULER = 2.71828182845904523536;
static const double PI    = 3.14159265358979323846;
```

A floating literal without a suffix has type double. Use f for float and L for long double. For scientific/simulation code, double is usually the sensible default. Hexadecimal floating-point literals exist in C but are mainly useful when exact binary floating-point representation is important.

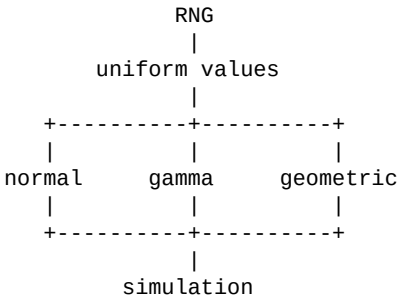
13. Probability distributions are not in standard math.h

Standard C math.h provides mathematical primitives, not probability distribution samplers. Functions such as normal, gamma, Poisson or negative-binomial sampling must be implemented separately or obtained from a library.

```
double random_normal(RNG *rng, double mean, double sd);
int    random_geometric(RNG *rng, double p);
double random_gamma(RNG *rng, double shape, double scale);
int    random_negative_binomial(RNG *rng, double r, double p);
```

14. RNG vs probability distribution

RNG and distribution are different layers. An RNG generates the basic pseudorandom sequence. Distribution algorithms transform that source into samples with the desired probability distribution.



15. PCG32

PCG32 is a small pseudorandom number generator. A stateful RNG object can be represented as:

```
typedef struct {
    uint64_t state;
    uint64_t inc;
} RNG;

uint32_t pcg32_random_r(RNG *rng);
```

Each call returns a new 32-bit pseudorandom integer and advances the RNG state. The caller should initialize the state properly before generating values. A seeding function is preferable to manually assigning arbitrary state/stream values.

```
uint32_t r1 = pcg32_random_r(&rng);
uint32_t r2 = pcg32_random_r(&rng);
uint32_t r3 = pcg32_random_r(&rng);
```

16. One RNG per thread / simulation

For parallel simulations, a separate RNG instance per independent execution context is a clean design. If each thread accesses only its own RNG, the mutable state is not shared and the RNG itself does not introduce a race condition.

```
Thread 0                Thread 1
   |                       |
   v                       v
RNG rng0                RNG rng1
state0                  state1
   |                       |
pcg32_random_r()        pcg32_random_r()
```

Do not have multiple threads update the same RNG instance without synchronization. For reproducible parallel work, give simulations controlled seeds and, where appropriate, distinct PCG streams.

17. Converting PCG32 output to U(0,1)

A uint32_t has 32 bits, so its possible values are 0 through 2^32-1. Since 2^32 = 4,294,967,296, a simple normalization is:

```
double u =
    (double)pcg32_random_r(rng) / 4294967296.0;
```

This produces one of 2^32 discrete values in [0,1). It is not literally a continuous uniform distribution, but this is the normal digital interpretation of a uniform pseudorandom source.

The cast to double matters because it ensures floating-point division. The denominator is 2^32, not UINT32_MAX, so the maximum result remains below 1.

For algorithms such as Box–Muller, avoid U1 = 0 because log(0) is invalid. It is useful to provide an RNG helper with an explicitly open interval or otherwise guarantee a strictly positive value where the distribution algorithm requires it.

18. Box–Muller: RNG → normal distribution

The formula shown in the discussion is the Box–Muller transform:

```
Z = sqrt(-2 ln(U1)) * cos(2 pi U2)
X = mu + sigma * Z
```

where U1 and U2 are independent uniform values in (0,1). The PCG32 RNG supplies those values; Box–Muller transforms them into a standard normal Z, and the second equation changes it to N(mu, sigma^2).

```
double random_normal(RNG *rng, double mu, double sigma)
{
    double u1 = rng_uniform_open(rng);
    double u2 = rng_uniform_open(rng);

    double z = sqrt(-2.0 * log(u1))
             * cos(2.0 * PI * u2);

    return mu + sigma * z;
}
```

Each Box–Muller sample in this straightforward form consumes two RNG outputs. Other normal-generation algorithms can trade mathematical simplicity for speed, but Box–Muller is clear and easy to validate.

19. Suggested project architecture

```
project/
■■■ Makefile
■■■ src/
■   ■■■ main.c
■   ■■■ agents/
■   ■   ■■■ agents.c
■   ■   ■■■ agents.h
■   ■■■ engine/
■   ■   ■■■ monte_carlo.c
■   ■   ■■■ monte_carlo.h
■   ■■■ random/
■   ■   ■■■ rng.c
■   ■   ■■■ rng.h
■   ■   ■■■ distributions.c
■   ■   ■■■ distributions.h
■   ■■■ utils/
■       ■■■ csv_parser.c
■       ■■■ csv_parser.h
■■■ build/
■■■ .vscode/
    ■■■ c_cpp_properties.json
```

A good separation is: RNG implementation → uniform helper → distribution samplers → simulation/agents. This keeps mathematical distribution logic independent of the particular pseudorandom generator.

20. Header include conventions

Use angle brackets for standard/system headers and quotes for project headers:

```
#include <stdint.h>
#include <math.h>
#include <stdlib.h>

#include "random/rng.h"
#include "random/distributions.h"
#include "engine/monte_carlo.h"
```

Rather than writing relative paths such as ../../engine/monte_carlo.h, configure the compiler's include path with -Isrc and include project headers relative to src.

21. Professional Makefile concepts

For a project of this size, one top-level Makefile is normally preferable to a Makefile in every directory. It can recursively discover .c files, compile them into a mirrored build/ tree, generate dependency files, link libm, and provide debug/release/run/clean targets.

```
CC       := gcc
CSTD     := c17
CPPFLAGS := -Isrc
CFLAGS   := -std=$(CSTD) -Wall -Wextra -Wpedantic
LDLIBS   := -lm

# Discover all .c files under src/
SOURCES  := $(shell find src -type f -name '*.c')
OBJECTS  := $(patsubst src/%.c,build/%.o,$(SOURCES))

simulation: $(OBJECTS)
■$(CC) $(OBJECTS) -o $@ $(LDLIBS)

build/%.o: src/%.c
■@mkdir -p $(dir $@)
■$(CC) $(CPPFLAGS) $(CFLAGS) -c $< -o $@
```

For a larger project, GCC's -MMD -MP options can generate header dependencies automatically, allowing Make to rebuild only the object files affected by header changes.

22. VS Code IntelliSense

```
{
    "configurations": [
        {
            "name": "Linux-GCC",
            "compilerPath": "/usr/bin/gcc",
            "cStandard": "c17",
            "intelliSenseMode": "linux-gcc-x64",
            "includePath": [
                "${workspaceFolder}/src"
            ]
        }
    ],
    "version": 4
}
```

The Makefile's -Isrc tells GCC where to search. c_cpp_properties.json tells VS Code's C/C++ extension the corresponding include path so IntelliSense does not flag valid project includes as errors. For larger projects, compile_commands.json can eventually provide the editor with the actual compiler commands and reduce duplicated configuration.

23. Recommended conventions for this project

• **Keep .h and .c together** when they belong to the same module.

• **Use one top-level Makefile** for the application unless separate build systems become genuinely necessary.

• **Use #pragma once** at the top of project headers.

• **Make headers self-contained**: include the headers they directly depend on.

• **Keep implementations in .c files**; headers normally expose declarations, types and small intentional inline/macros.

• **Do not name a project header math.h**; use a more specific name such as math_utils.h.

• **Use double** as the default precision for numerical simulation unless there is a concrete reason to use float.

• **Separate RNG from distributions**: PCG32 supplies pseudorandomness; distribution algorithms transform it.

• **Use one RNG state per independent simulation/thread** to avoid shared mutable RNG state and to make reproducibility easier.

• **Use descriptive module names** rather than allowing utils/ to become an uncontrolled miscellaneous directory.